

Name: Ayesha Mudassar

Roll No: 232471

Section: BSCSev-6B

Subject: Advanced Database Management System

Assignment Scenario: Intelligent Distributed Data Platform

1. Introduction

Modern digital platforms generate extremely large amounts of data every second. Applications such as e-commerce platforms, social media systems, recommendation engines, and analytics systems require databases capable of handling millions of users and transactions simultaneously.

Traditional relational databases face several limitations when dealing with:

- Very large distributed datasets
- Dynamic and evolving schemas
- Real-time processing requirements
- High availability across multiple regions
- Horizontal scalability

To overcome these limitations, organizations are increasingly adopting distributed NoSQL architectures. NoSQL databases provide flexible schemas, distributed storage, fault tolerance, and high-performance processing.

This report presents the design of an **Intelligent Distributed Data Platform** that integrates:

- Online marketplace services
- Social networking features
- Real-time analytics and recommendations
- Multi-region distributed deployment

The proposed solution focuses on building a scalable, secure, and highly available distributed NoSQL-based architecture.

2. System Requirements and Challenges

The system must support multiple functionalities while maintaining high performance and reliability.

2.1 Functional Requirements

The platform must support:

Online Marketplace

- Product management
- Orders and transactions
- Payment processing
- Inventory tracking

Social Features

- Reviews and ratings
- Comments and likes
- User relationships and connections

Analytics Features

- Real-time analytics
- Recommendation engine
- User activity tracking

Multi-Region Support

- Distributed deployment
- Data replication
- Global access

2.2 System Challenges

The organization faces the following challenges:

Challenge	Description
Massive Data Growth	Continuous increase in user and transaction data
Flexible Data Structures	Different entities require different schemas
High Availability	System must remain online with minimal downtime
High Availability	Communication failures between regions
Security	Sensitive user and payment information protection
Scalability	Ability to support millions of users

3. Task 1 – Data Architecture Design

3.1 Major Categories of Data

The system contains several categories of data with different storage requirements.

3.1.1 User Data

This includes:

- User profiles
- Login credentials
- Addresses
- Preferences
- Purchase history

Characteristics

- Frequently accessed
- Semi-structured
- Dynamic fields

Selected NoSQL Model

Document-Oriented Database (MongoDB)

Justification

MongoDB supports flexible schemas and nested documents which are suitable for user profiles with varying structures.

3.1.2 Product Data

This includes:

- Product descriptions
- Product categories
- Pricing
- Inventory
- Specifications

Characteristics

- Dynamic attributes
- Large product catalog
- High read operations

Selected NoSQL Model

Document Database

Justification

Different products have different specifications. NoSQL databases easily support varying fields without schema redesign.

3.1.3 Order and Payment Data

This includes:

- Orders
- Payment transactions
- Billing details
- Shipping records

Characteristics

- High write operations
- Requires reliability
- Distributed storage required

Selected NoSQL Model

Document Database with Replication

Justification

Order information naturally fits into nested document structures.

3.1.4 Social Interaction Data

This includes:

- Reviews
- Ratings
- Comments

- Likes
- Followers

Characteristics

- Relationship-intensive
- Highly connected data
- Real-time interactions

Selected NoSQL Model

Graph Database + Document Store

Justification

Graph databases efficiently manage relationships between users and products.

3.1.5 Analytics Data

This includes:

- Clickstream data
- Logs
- Monitoring metrics
- Recommendation events

Characteristics

- Huge volume
- Continuous writes
- Real-time processing

Selected NoSQL Model

Wide Column Store (Cassandra)

Justification

Column-based databases support fast distributed writes and analytical processing.

3.2 High-Level Data Architecture

The proposed architecture contains the following layers:

1. Client Layer
2. API Gateway Layer
3. Microservices Layer
4. NoSQL Database Layer
5. Real-Time Analytics Layer
6. Monitoring and Security Layer

3.3 Architecture Justification

Requirement	Solution
Flexibility	Schema-less NoSQL databases
Scalability	Horizontal scaling using distributed nodes
High Availability	Replication across regions
Performance	Optimized access patterns
Fault Tolerance	Distributed replication and failover
Real-Time Processing	Stream-based analytics

4. Task 2 – Conceptual Data Modeling

4.1 MongoDB Collections Design

USERS Collection

```
{
  "_id": "U1001",
  "name": "Ali Khan",
  "email": "ali@gmail.com",
  "password": "hashed_password",
  "phone": "03001234567",
  "addresses": [
    {
      "city": "Rawalpindi",
      "zip": "46000"
    }
  ],
  "preferences": {
```

```
    "language": "English",
    "currency": "PKR"
  }
}
```

PRODUCTS Collection

```
{
  "_id": "P1001",
  "name": "Gaming Laptop",
  "category": "Electronics",
  "brand": "Dell",
  "price": 250000,
  "stock": 20,
  "specifications": {
    "ram": "16GB",
    "storage": "1TB SSD"
  }
}
```

ORDERS Collection

```
{
  "_id": "O1001",
  "user_id": "U1001",
  "items": [
    {
      "product_id": "P1001",
      "quantity": 1,
      "price": 250000
    }
  ],
  "total_amount": 250000,
  "payment_status": "Paid",
  "status": "Shipped"
}
```

REVIEWS Collection

```
{
  "_id": "R1001",
  "user_id": "U1001",
  "product_id": "P1001",
  "rating": 5,
}
```



```
"comment": "Excellent product"
}
```

4.2 Embedded Documents

Embedded documents are used when related data is frequently accessed together.

Examples

- User addresses inside Users collection
- Product specifications inside Products collection
- Order items inside Orders collection

Advantages

- Faster retrieval
- Fewer joins
- Improved performance

4.3 Referenced Documents

Referenced documents are used when data relationships are large or reusable.

Examples

- user_id in Orders
- product_id in Reviews

Advantages

- Reduced duplication
- Better scalability
- Easier updates

4.4 Data Model Design Goals

The data model is designed to ensure:

- Efficient querying
- High performance
- Distributed storage support

- Reduced redundancy
- Scalability

5. Task 3 – Distributed Design using CAP Theorem

5.1 CAP Theorem Overview

The CAP theorem states that a distributed system can only guarantee two of the following three properties simultaneously:

Property	Description
Consistency	All nodes contain the same data
Availability	System always responds
Partition Tolerance	System continues during network failures

5.2 Selected CAP Model

The proposed system prioritizes:

Availability + Partition Tolerance (AP)

5.3 Justification

The platform operates globally across multiple regions. Network failures may occur between distributed data centers.

Therefore:

- The system must remain available
- Users should continue accessing services
- Temporary inconsistency is acceptable

This approach supports:

- Continuous operation
- High user experience
- Fault tolerance

5.4 Trade-Offs

The system uses eventual consistency.

Example

If stock quantity changes in one region, another region may temporarily display older data.

However:

- System remains online
- User operations continue successfully

6. Task 4 – Data Interaction and Query Design

6.1 Data Insertion Query

```
db.users.insertOne({  
  _id: "U1002",  
  name: "Sara Ahmed",  
  email: "sara@gmail.com",  
  phone: "03111222333"  
});
```

6.2 Data Retrieval Query

```
db.products.find(  
  {  
    name: "Sara Ahmed"  
  }  
);
```

```
{ category: "Electronics" },  
{ name: 1, price: 1, stock: 1 }  
);
```

6.3 Data Update Query

```
db.orders.updateOne(  
  { _id: "O1001" },  
  {  
    $set: {  
      status: "Delivered"  
    }  
  }  
);
```

6.4 Data Deletion Query

```
db.reviews.deleteOne({  
  _id: "R1001"  
});
```

6.5 Aggregation Query

```
db.orders.aggregate([  
  {  
    $group: {  
      _id: "$user_id",  
      total_spent: {  
        $sum: "$total_amount"  
      }  
    }  
  },  
  {  
    $sort: {  
      total_spent: -1  
    }  
  }  
]);
```

6.6 Query Design Considerations

The queries are optimized for:

- Fast execution
- Large-scale datasets

- Real-time processing
- Reduced database load

Techniques used include:

- Indexing
- Projection
- Aggregation pipelines
- Sharding

7. Task 5 – Scalability and Performance Strategy

7.1 Horizontal Scalability

The system uses horizontal scaling by adding more distributed nodes.

Advantages

- Unlimited growth support
- Improved performance
- Reduced server overload

7.2 Sharding Strategy

Sharding distributes data across multiple servers.

Shard Keys

Collection	Shard Key
Users	user_id
Products	category
Orders	order_id

7.3 Load Balancing

Load balancers distribute traffic evenly across servers.

Benefits

- Prevents bottlenecks
- Improves response time
- Ensures reliability

7.4 Indexing Strategy

Indexes are created on:

- user_id
- product_id
- category
- email
- order_date

Benefits

- Faster searching
- Improved query execution
- Reduced scan operations

7.5 Query Optimization Techniques

The system uses:

- Aggregation pipelines
- Query projection
- Caching
- Optimized indexes
- Denormalization

These techniques improve:

- Performance
- Throughput
- Scalability

8. Task 6 – Security and Reliability Design

8.1 Data Confidentiality

Sensitive data is protected using:

- AES-256 encryption
- TLS/HTTPS communication

8.2 Access Control

Role-Based Access Control (RBAC) is implemented.

Roles

- Admin
- Customer
- Analyst

Each role has limited permissions.

8.3 Authentication

Authentication methods include:

- JWT Tokens
- OAuth Authentication
- Multi-factor Authentication

8.4 Fault Tolerance

The system ensures fault tolerance through:

- Replication
- Automatic failover
- Distributed backups

8.5 System Recovery

Recovery mechanisms include:

- Snapshot backups
- Multi-region replication
- Automated recovery systems

These mechanisms reduce downtime and prevent data loss.

9. Task 7 – Design Evaluation and Justification

9.1 Strengths of Proposed Design

Strength	Description
Scalability	Supports millions of users
Flexibility	Dynamic schema support
High Availability	Continuous operation
Fault Tolerance	Handles node failures
Real-Time Analytics	Supports instant insights
Performance	Optimized distributed queries

9.2 Limitations and Trade-Offs

Limitation	Explanation
Eventual Consistency	Data synchronization delay
Increased Complexity	Distributed systems are harder to manage
Data Duplication	Denormalization increases storage

9.3 When Relational Databases Are Better

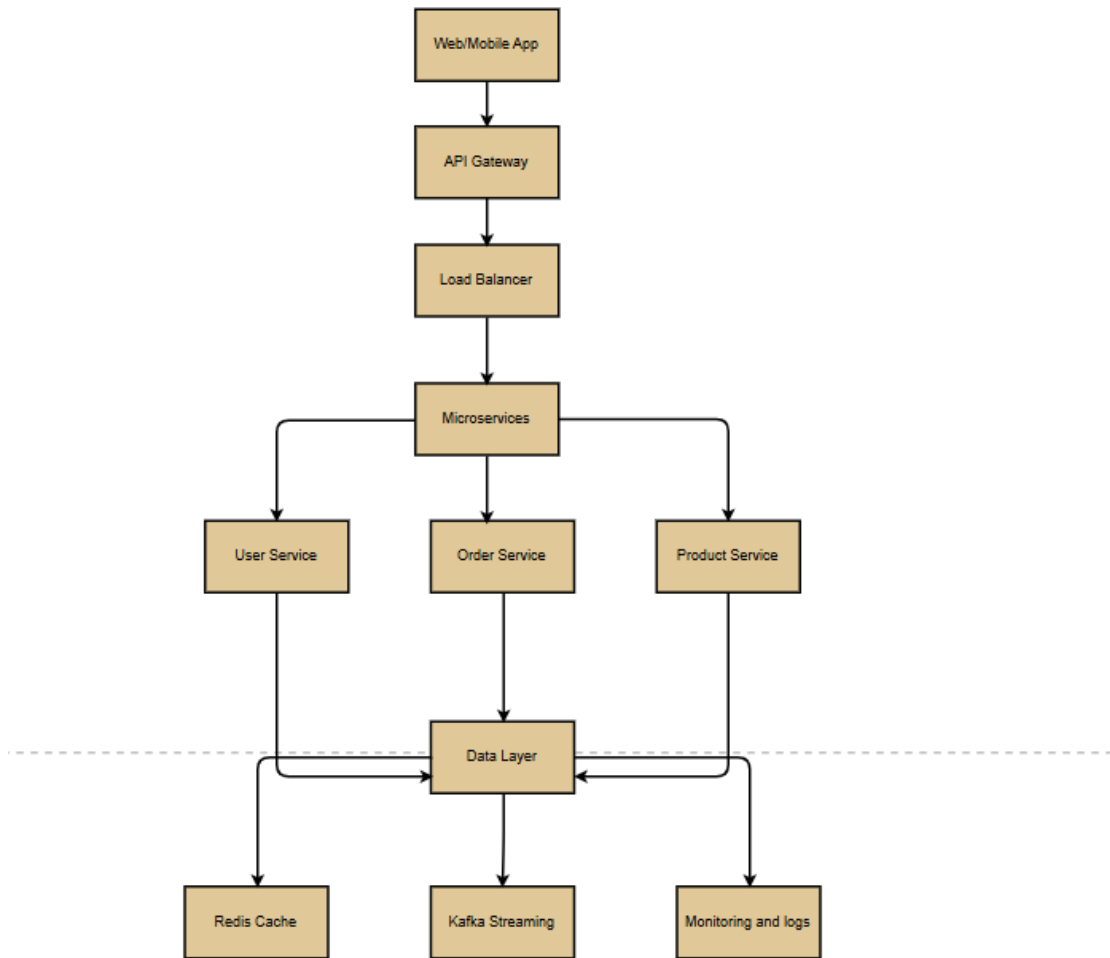
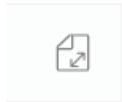
Relational databases remain suitable for:

- Banking systems
- Financial systems
- Airline reservation systems
- Applications requiring strict ACID compliance

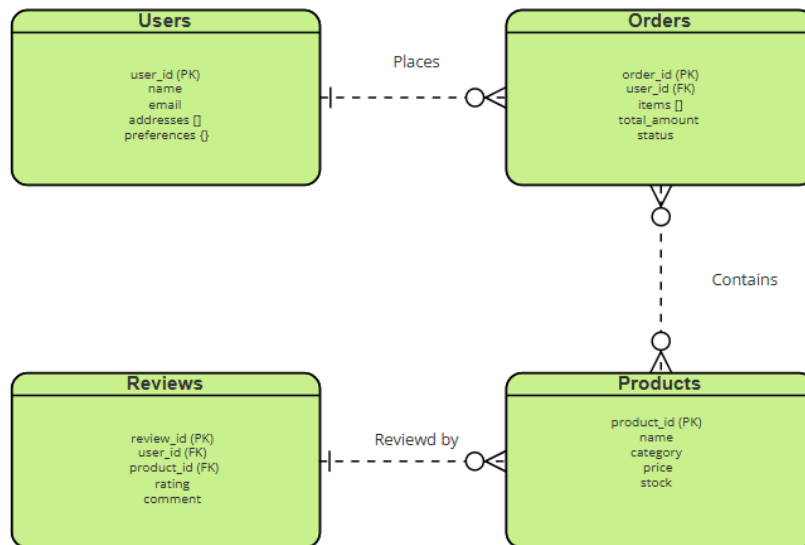
These systems require:

- Strong consistency
- Complex joins
- Structured schemas

10. System Architecture Diagram



11. Data Model Diagram



12. Outputs

Create Tables

pgAdmin 4

File Object Tools Edit View Window Help

postgres/postgres@PostgreSQL 18* X

postgres/postgres@PostgreSQL 18

Query Query History

```

1 CREATE TABLE users (
2     user_id VARCHAR(10) PRIMARY KEY,
3     name VARCHAR(100),
4     email VARCHAR(100) UNIQUE,
5     password VARCHAR(100),
6     phone VARCHAR(20),
7     city VARCHAR(50),
8     zip_code VARCHAR(10),
9     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
10 );
11
12 CREATE TABLE products (
13     product_id VARCHAR(10) PRIMARY KEY,
14     name VARCHAR(100),
15     category VARCHAR(50),
16     brand VARCHAR(50),
17     price NUMERIC(10,2),
18     stock INT,
19     ram VARCHAR(20),
20     storage VARCHAR(50)
  
```

Data Output Messages Notifications

CREATE TABLE

Query returned successfully in 2 secs 216 msec.

Insert Data

pgAdmin 4

File Object Tools Edit View Window Help

postgres/postgres@PostgreSQL 18*

postgres/postgres@PostgreSQL 18

Query Query History

```
69 INSERT INTO users
70 VALUES
71 ('U1001','Ali Khan','ali@gmail.com','pass123','03001234567','Rawalpindi','46000',CURRENT_TIMESTAMP),
72
73 ('U1002','Sara Ahmed','sara@gmail.com','pass456','03111222333','Islamabad','44000',CURRENT_TIMESTAMP);
74
75 INSERT INTO products
76 VALUES
77 ('P1001','Gaming Laptop','Electronics','Dell',250000,20,'16GB','1TB SSD'),
78
79 ('P1002','iPhone 15','Mobile','Apple',350000,15,'8GB','256GB');
80
81 INSERT INTO orders
82 VALUES
83 ('O1001','U1001',250000,'Paid','Shipped',CURRENT_TIMESTAMP),
84
85 ('O1002','U1002',350000,'Paid','Processing',CURRENT_TIMESTAMP);
86
87 INSERT INTO order_items(order_id,product_id,quantity,price)
88 VALUES
```

Data Output Messages Notifications

INSERT 0 2

Query returned successfully in 146 msec.

Select Queries

View Users

pgAdmin 4

File Object Tools Edit View Window Help

postgres/postgres@PostgreSQL 18*

postgres/postgres@PostgreSQL 18

Query Query History

```
87 INSERT INTO order_items(order_id,product_id,quantity,price)
88 VALUES
89 ('O1001','P1001',1,250000),
90
91 ('O1002','P1002',1,350000);
92
93 INSERT INTO reviews
94 VALUES
95 ('R1001','U1001','P1001',5,'Excellent Product',10),
96
97 ('R1002','U1002','P1002',4,'Very Good Phone',5);
98
99 SELECT * FROM users;
```

Data Output Messages Notifications

	user_id [PK] character varying (10)	name character varying (100)	email character varying (100)	password character varying (100)	phone character varying (20)	city character varying (50)	zip_code character varying (10)	created_at timestamp without time zone
1	U1001	Ali Khan	ali@gmail.com	pass123	03001234567	Rawalpindi	46000	2026-05-16 22:01:02.210113
2	U1002	Sara Ahmed	sara@gmail.com	pass456	03111222333	Islamabad	44000	2026-05-16 22:01:02.210113

View Products

pgAdmin 4

File Object Tools Edit View Window Help

postgres/postgres@PostgreSQL 18*

postgres/postgres@PostgreSQL 18

No limit

Query Query History Scratch Pad

```
89 ('O1001','P1001',1,250000),
90
91 ('O1002','P1002',1,350000);
92
93 INSERT INTO reviews
94 VALUES
95 ('R1001','U1001','P1001',5,'Excellent Product',10),
96
97 ('R1002','U1002','P1002',4,'Very Good Phone',5);
98
99 SELECT * FROM users;
100
101 SELECT * FROM products;
```

Data Output Messages Notifications

Showing rows: 1 to 2 Page No: 1

	product_id [PK] character varying (10)	name character varying (100)	category character varying (50)	brand character varying (50)	price numeric (10,2)	stock integer	ram character varying (20)	storage character varying (50)
1	P1001	Gaming Laptop	Electronics	Dell	250000.00	20	16GB	1TB SSD
2	P1002	iPhone 15	Mobile	Apple	350000.00	15	8GB	256GB

View Orders with User Names

pgAdmin 4

File Object Tools Edit View Window Help

postgres/postgres@PostgreSQL 18*

postgres/postgres@PostgreSQL 18

No limit

Query Query History

```
98
99 SELECT * FROM users;
100
101 SELECT * FROM products;
102
103 SELECT
104     o.order_id,
105     u.name,
106     o.total_amount,
107     o.status
108 FROM orders o
109 JOIN users u
110 ON o.user_id = u.user_id;
```

Data Output Messages Notifications

Showing rows: 1 to 2

	order_id character varying (10)	name character varying (100)	totalAmount numeric (10,2)	status character varying (20)
1	O1001	Ali Khan	250000.00	Shipped
2	O1002	Sara Ahmed	350000.00	Processing

Update Query

pgAdmin 4

File Object Tools Edit View Window Help

postgres/postgres@PostgreSQL 18* X

postgres/postgres@PostgreSQL 18

No limit

Query Query History

```
102
103 SELECT
104     o.order_id,
105     u.name,
106     o.total_amount,
107     o.status
108 FROM orders o
109 JOIN users u
110 ON o.user_id = u.user_id;
111
112 UPDATE orders
113 SET status = 'Delivered'
114 WHERE order_id = '01001';
```

Data Output Messages Notifications

UPDATE 1

Query returned successfully in 147 msec.

Delete Query

pgAdmin 4

File Object Tools Edit View Window Help

postgres/postgres@PostgreSQL 18* X

postgres/postgres@PostgreSQL 18

No limit

Query Query History

```
105     u.name,
106     o.total_amount,
107     o.status
108 FROM orders o
109 JOIN users u
110 ON o.user_id = u.user_id;
111
112 UPDATE orders
113 SET status = 'Delivered'
114 WHERE order_id = '01001';
115
116 DELETE FROM reviews
117 WHERE review_id = 'R1002';
```

Data Output Messages Notifications

DELETE 1

Query returned successfully in 80 msec.

Aggregate Queries

Total Sales

pgAdmin 4

File Object Tools Edit View Window Help

postgres/postgres@PostgreSQL 18* X

postgres/postgres@PostgreSQL 18

Query Query History

```
108 FROM orders o
109 JOIN users u
110 ON o.user_id = u.user_id;
111
112 UPDATE orders
113 SET status = 'Delivered'
114 WHERE order_id = 'O1001';
115
116 DELETE FROM reviews
117 WHERE review_id = 'R1002';
118
119 SELECT SUM(total_amount) AS total_sales
120 FROM orders;
```

Data Output Messages Notifications

Showing rows

	total_sales	numeric
1	600000.00	

Total Orders Per User

pgAdmin 4

File Object Tools Edit View Window Help

postgres/postgres@PostgreSQL 18* X

postgres/postgres@PostgreSQL 18

Query Query History

```
114 WHERE order_id = 'O1001';
115
116 DELETE FROM reviews
117 WHERE review_id = 'R1002';
118
119 SELECT SUM(total_amount) AS total_sales
120 FROM orders;
121
122 SELECT
123     user_id,
124     COUNT(order_id) AS total_orders
125 FROM orders
126 GROUP BY user_id;
```

Data Output Messages Notifications

	user_id	character varying (10)	total_orders	bigint
1	U1001		1	
2	U1002		1	

Average Product Rating

pgAdmin 4

File Object Tools Edit View Window Help

postgres/postgres@PostgreSQL 18* X

postgres/postgres@PostgreSQL 18

No limit

Query Query History

```
120 FROM orders;
121
122 SELECT
123     user_id,
124     COUNT(order_id) AS total_orders
125 FROM orders
126 GROUP BY user_id;
127
128 SELECT
129     product_id,
130     AVG(rating) AS average_rating
131 FROM reviews
132 GROUP BY product_id;
```

Data Output Messages Notifications

	product_id character varying (10)	average_rating numeric
1	P1001	5.0000000000000000

Indexing Strategy

pgAdmin 4

File Object Tools Edit View Window Help

postgres/postgres@PostgreSQL 18* X

postgres/postgres@PostgreSQL 18

No limit

Query Query History

```
129     product_id,
130     AVG(rating) AS average_rating
131 FROM reviews
132 GROUP BY product_id;
133
134 CREATE INDEX idx_user_email
135 ON users(email);
136
137 CREATE INDEX idx_product_category
138 ON products(category);
139
140 CREATE INDEX idx_order_user
141 ON orders(user_id);
```

Data Output Messages Notifications

CREATE INDEX

Query returned successfully in 134 msec.

13. Conclusion

This report proposed a scalable distributed NoSQL architecture for a modern intelligent digital platform. The system supports marketplace services, social interactions, analytics processing, and multi-region deployment.

The proposed architecture successfully addresses:

- Scalability challenges
- Distributed storage requirements
- High availability
- Fault tolerance
- Security concerns
- Real-time analytics

By using NoSQL technologies such as MongoDB, Cassandra, and graph databases, the platform achieves high flexibility and performance suitable for modern large-scale applications.